Zuul Project Report: Pokémon Delta Emerald

Site Web: https://perso.esiee.fr/~jouanarb/

Groupe: A3P 2025/2026 G5

I.A) Auteur : Barnabe Jouanard

I.B) Thème

Un jeu d'aventure se déroulant dans la région de Hoenn (Pokémon), axé sur l'exploration et la résolution d'énigmes environnementales pour mettre fin à un conflit légendaire entre la terre et la mer.

I.C) Résumé du Scénario

Le joueur incarne un dresseur à Littleroot Town. Le monde subit un climat anormal. L'objectif principal est de traverser la partie ouest de Hoenn pour trouver l'Orbe Émeraude (cachée à Petalburg City) et l'apporter au Pilier Céleste pour réveiller Rayquaza.

I.D) Full Plan

La carte se compose de 9 lieux principaux reliés ainsi :

Your House: Starting location.

Littleroot Town: Connecting hub.

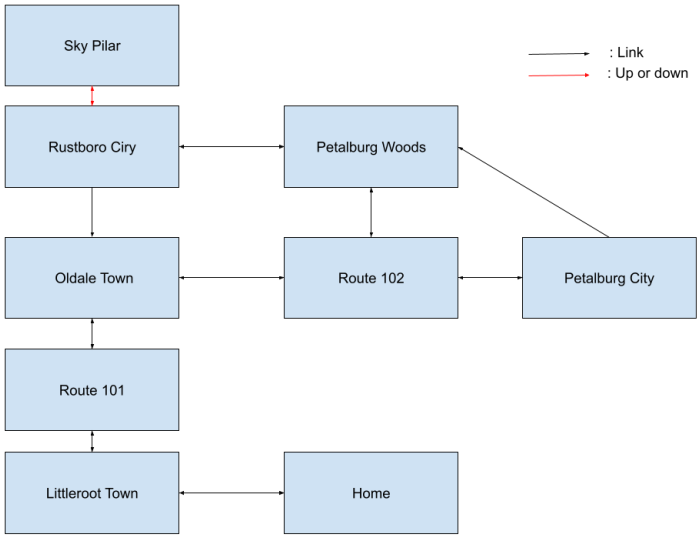
Route 101: Path to the north.

Oldale Town: Junction to the west.

Route 102: Combat/Trainer route.

Petalburg City: City containing a locked Gym.

Sky Pillar: The final destination.



I.E) Scenario Detaille

Le joueur commence dans sa Maison. Il doit se rendre à Bourg-en-Vol puis sur la Route 101 pour rencontrer le Professeur Seko. Après avoir reçu des instructions, il se dirige vers Bourg-en-Paume et Clémenti-Ville. Le joueur devra trouver un chemin à travers les Bois Clémenti pour atteindre le Pilier Céleste.

I.F) Details of Locations, Items, and Characters

Personnages : Professeur Seko (donne des indices), Norman (Champion d'arène à Rustboro City).

Objets : - Chaussures : (Futur) pour se déplacer plus vite.

- Map : Pour voir la carte.
- Delta Orb : (Futur) pour invoquer Rayquazza

I.G) Conditions de Victoire/Défaite

Victoire : Atteindre le sommet du Pilier Céleste en possédant l'Orbe Émeraude.

Défaite : Un compteur de tours simulant l'aggravation de la météo. Si le compteur atteint zéro, la région est détruite. (a venir)

I.I) Commentaires

La version 2 n'a pour l'instant aucune histoire, mais elle commence a ajouter petits a petits des fonctionnalités essentielles pour la suite et elle a une condition de victoire.

II. Réponses aux Exercices

Exercice 7.5

J'ai implémenté la méthode privée printLocationInfo() dans Game pour centraliser l'affichage de la description de la pièce et des sorties. Cela évite la duplication de code entre printWelcome() et goRoom().

```
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print("Exits : ");
    if (this.aCurrentRoom.aNorthExit != null){
        System.out.print("North ");
    }
    if (this.aCurrentRoom.aSouthExit != null){
        System.out.print("South ");
    }
    if (this.aCurrentRoom.aEastExit != null){
        System.out.print("East ");
    }
    if (this.aCurrentRoom.aWestExit != null){
        System.out.print("West ");
    }
}
```

Exercice 7.6 :

J'ai passé les attributs de la classe Room en private et créé l'accesseur getExit pour masquer les détails internes des sorties. Cette modification réduit le couplage en permettant à la classe Game de demander une direction sans accéder directement aux variables de Room.

```java
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

```java
public Room getExit(final String pDirection){
    if (pDirection.equals("North"))
    {
        return aNorthExit;
    }
    if (pDirection.equals("South"))
    {
        return aSouthExit;
    }
    if (pDirection.equals("East"))
    {
        return aEastExit;
    }
    if (pDirection.equals("West"))
    {
        return aWestExit;
    }
    return null;
}
```

Exercice 7.7 :

J'ai créé la méthode getExitString dans Room pour déléguer la préparation de la liste des sorties à la pièce elle-même. Cela permet à Game d'afficher les informations sans avoir à connaître la structure interne des sorties de Room.

```java
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.getExit("North") != null) {
    vReturnString += " North";
    }

    if (this.getExit("East") != null) {
    vReturnString += " East";
    }

    if (this.getExit("South") != null) {
    vReturnString += " South";
    }

    if (this.getExit("West") != null) {
    vReturnString += " West";
    }

    return vReturnString;
} // getExitString()
```

```java
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print(this.aCurrentRoom.getExitString());
    System.out.println();
}
```

Exercice 7.8 :

J'ai remplacé les attributs de direction par une HashMap<String, Room> pour centraliser la gestion des sorties. J'ai adapté getExitString, printLocationInfo, getExit et le constructeur pour HashMap et aussi réécrit dans la classe game les sorties pour adapter ces changements.

```
private HashMap<String, Room> aExits;
```

```
public Room (final String pDescription)
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
} // Room()
```

```
public Room getExit(final String pDirection)
{
    return this.aExits.get(pDirection);
} // getExit()
```

```
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.aExits.get("North") != null) {
    vReturnString += " North";
    }

    if (this.aExits.get("East") != null) {
    vReturnString += " East";
    }

    if (this.aExits.get("South") != null) {
    vReturnString += " South";
    }

    if (this.aExits.get("West") != null) {
    vReturnString += " West";
    }
```

```
public void setExit (final String pDirection,final Room pExit)
{
    this.aExits.put(pDirection, pExit);
} // setExits()
```

```
    vHouse.setExit("West", vLittleroot);

    vLittleroot.setExit("North", vRoute101);
    vLittleroot.setExit("East", vHouse);

    vRoute101.setExit("North", vOldale);
    vRoute101.setExit("South", vLittleroot);

    vOldale.setExit("East", vRoute102);
    vOldale.setExit("South", vRoute101);

    vRoute102.setExit("North", vPetalburgWoods);
    vRoute102.setExit("East", vPetalburg);
    vRoute102.setExit("West", vOldale);

    vPetalburg.setExit("South", vRoute102);
    vPetalburg.setExit("West", vRustboro);

    vPetalburgWoods.setExit("South", vRoute102);
    vPetalburgWoods.setExit("West", vRustboro);
```

Exercice 7.8.1 :

J'ai étendu la gestion des déplacements en ajoutant les directions verticales "up" et "down", permettant ainsi d'accéder au Sky Pillar. Pour cela, j'ai simplement utilisé la nouvelle méthode setExit pour lier les pièces et mis à jour getExitString pour afficher ces sorties supplémentaires ainsi que goRoom pour pouvoir se déplacer.

```java
public String getExitString()
{
    String vReturnString = "Exits:";

    if (this.aExits.get("North") != null) {
    vReturnString += " North";
    }

    if (this.aExits.get("East") != null) {
    vReturnString += " East";
    }

    if (this.aExits.get("South") != null) {
    vReturnString += " South";
    }

    if (this.aExits.get("West") != null) {
    vReturnString += " West";
    }

    if (this.aExits.get("Down") != null) {
    vReturnString += " Down";
    }

    if (this.aExits.get("Up") != null) {
    vReturnString += " Up";
    }

    return vReturnString;
} // getExitString()
```

```java
        else if (pCommand.getSecondWord().equals("Up"))
        {
            if (this.aCurrentRoom.getExit("Up")!=null)
            {
                this.aCurrentRoom=this.aCurrentRoom.getExit("Up");
                this.printLocationInfo();
            } else
            {
                System.out.println("There is no door !");
            }
        }
        else if (pCommand.getSecondWord().equals("Down"))
        {
            if (this.aCurrentRoom.getExit("Down")!=null)
            {
                this.aCurrentRoom=this.aCurrentRoom.getExit("Down");
                this.printLocationInfo();
            } else
            {
                System.out.println("There is no door !");
            }
        }
```

```java
    vRustboro.setExit("Up", vSkyPillar);
    vRustboro.setExit("East", vPetalburgWoods);

    vSkyPillar.setExit("Down", vRustboro);
```

Exercice 7.9 :

J'ai remplacé la vérification manuelle des directions dans getExitString par une boucle for-each parcourant l'ensemble des clés (keySet) de la HashMap . Cette modification permet de construire dynamiquement la liste des sorties disponibles, rendant la classe Room capable d'afficher n'importe quelle nouvelle direction (comme "up" ou "down") sans aucune modification de code supplémentaire.

```java
public String getExitString()
{
    String vReturnString = "Exits:";
    Set<String> vKeys = this.aExits.keySet(); // Récupère toutes les directions présentes
    for (String vExit : vKeys) {
        vReturnString += " " + vExit;
    } // for
    return vReturnString;
} // getExitString()
```

Exercice 7.11 :

J'ai implémenté la méthode getLongDescription() dans la classe Room afin qu'elle assemble elle-même la description du lieu et la liste des sorties. La méthode printLocationInfo() de la classe Game a été simplifiée pour n'appeler que cette fonction, masquant ainsi totalement les détails d'implémentation interne de la pièce au reste de l'application .

```
public String getLongDescription()
{
    return "You are " + this.getDescription() + '\n' + this.getExitString();
}
private void printLocationInfo()
{
    System.out.println("You are " + this.aCurrentRoom.getLongDescription());
} // printLocationInfo()
```

Exercice 7.14 :

J'ai créé la commande look qui permet au joueur d'afficher la description complète du lieu actuel et de ses sorties sans effectuer de déplacement. Pour améliorer l'expérience utilisateur, j'ai également mis à jour la liste des mots de commande dans CommandWords afin que look apparaisse lors de l'appel à la commande help.

```
private final String[] aValidCommands = { "go", "help", "quit", "look" };
```

```
    else if (pCommand.getCommandWord().equals("look")){
        look();
        return false;
```

```
private void look()
{
    System.out.println(this.aCurrentRoom.getLongDescription());
} //look()
```

```
private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println("  go quit help look");
} // printHelp()
```

Exercice 7.15 :

J'ai ajouté la commande eat qui, une fois activée, affiche un message indiquant que le joueur a mangé et n'a plus faim. Techniquement, cette commande a été enregistrée dans la classe CommandWords et intégrée à la structure de contrôle de la méthode processCommand dans la classe Game.

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat" };
```

```
    else if (pCommand.getCommandWord().equals("eat")){
        eat();
        return false;
```

```
private void eat()
{
    System.out.println("You have eaten now and you are not hungry any more.");
} //eat()
```

```java
private void printHelp()
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println("  go quit help look eat");
} // printHelp()
```

Exercice 7.16 :

J'ai délégué la responsabilité de l'affichage des commandes à la classe CommandWords via une méthode showAll(), appelée par le Parser. Ce changement permet à la commande help de mettre à jour automatiquement sa liste de mots (incluant look et eat) sans qu'aucune modification ne soit nécessaire dans la classe Game.

```java
public void showAll(){
    for(String vCommand : aValidCommands){
        System.out.print(vCommand + ' ');
    }
    System.out.println();
} // showAll()
```

```java
public void showCommands(){
    this.aValidCW.showAll();
} // showCommands()
```

```java
private void printHelp()
{
    System.out.println("You are in the wonderful word of Pokemon.");
    System.out.println("You are trying to stop Rayquazza from destroying Hoenn.");
    System.out.println("Your command words are:");
    aParser.showCommands();
} // printHelp()
```

Exercice 7.18:

La méthode showAll a été modifiée en getCommandList pour renvoyer une chaîne de caractères contenant les commandes, et les méthodes showCommand (Parser) et printHelp (Game) ont été adaptées en conséquence. Suite à une comparaison avec "Zuul better", la classe StringBuilder a été intégrée aux méthodes getExitString et getCommandList pour optimiser la construction des chaînes de caractères, et des problèmes de formatage ont été corrigés.

```java
public String getCommandList(){
    String vChaineDesCommandes = "";
    for(String vCommand : aValidCommands){
        vChaineDesCommandes = vChaineDesCommandes + vCommand + " ";
    }
    return vChaineDesCommandes;
} // showAll()
```

```java
public String getCommandsList()
{
    return this.aValidCW.getCommandList();
} // showCommands()
```

```java
private void printHelp()
{
    System.out.println("You are in the wonderful word of Pokemon.");
    System.out.println("You are trying to stop Rayquaza from destroying Hoenn.");
    System.out.println("Your command words are:");
    System.out.println(aParser.getCommandsList());
} // printHelp()
```

Exercice 7.18.1 a 7.18.4:

Le titre du jeu reste "Pokémon Delta Emerald". Des images ont été choisies pour les différentes pièces du jeu. La classe Room a été mise à jour avec un attribut aImageName et un accesseur getImageName() pour gérer ces illustrations.

```java
public String getExitString()
{
    StringBuilder vReturnString = new StringBuilder("Exits:");
    for (String vExit : this.aExits.keySet()) {
        vReturnString.append(" ").append(vExit);
    }
    return vReturnString.toString();
} // getExitString()
```

```java
public String getCommandList()
{
    StringBuilder vChaineDesCommandes = new StringBuilder();
    for (String vCommand : this.aValidCommands) {
        vChaineDesCommandes.append(vCommand).append(" ");
    }
    return vChaineDesCommandes.toString();
} // getCommandList()
```

Exercice 7.18.6:

le projet est passé sur la base "zuul with image", ce qui a impliqué plusieurs changements et adaptations dans le code. Par exemple, la classe Room a été adaptée avec l'ajout de l'attribut aImageName, de l'accesseur getImageName(), et la modification du constructeur pour prendre en charge et afficher ces illustrations.

```java
private String aImageName;
```

```java
public String getImageName()
{
    return this.aImageName;
} // getImageName()
```

```java
public Room (final String pDescription, final String pImage)
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
} // Room()
```

```java
public String getImageName()
{
    return this.aImageName;
} // getImageName()
```

```java
private CommandWords aValidCW;  // (voir la classe CommandWords)

/**
 * Constructeur par defaut qui cree les 2 objets prevus pour les attributs
 */
public Parser()
{
    this.aValidCW = new CommandWords();
    // System.in designe le clavier, comme System.out designe l'ecran
} // Parser()

/**
 * Get a new command from the user. The command is read by
 * parsing the 'inputLine'.
 */
public Command getCommand( final String pInputLine )
{
    String vWord1;
    String vWord2;

    StringTokenizer tokenizer = new StringTokenizer( pInputLine );

    if ( tokenizer.hasMoreTokens() )
        vWord1 = tokenizer.nextToken();      // get first word
    else
        vWord1 = null;

    if ( tokenizer.hasMoreTokens() )
        vWord2 = tokenizer.nextToken();      // get second word
    else
        vWord2 = null;

    // note: we just ignore the rest of the input line.

    // Now check whether this word is known. If so, create a command
    // with it. If not, create a "null" command (for unknown command).

    if ( this.aValidCW.isCommand( vWord1 ) )
        return new Command( vWord1, vWord2 );
    else
        return new Command( null, vWord2 );
} // getCommand()

/**
 * Print out a list of valid command words.
 */
public String getCommandString()
{
    return this.aValidCW.getCommandList();
} // getCommandString()
```

```java
private UserInterface aGui;
private GameEngine aEngine;

/**
 * Default constructor: initializes the engine and the gui.
 */
public Game()
{
    this.aEngine = new GameEngine();
    this.aGui = new UserInterface( this.aEngine );
    this.aEngine.setGUI( this.aGui );
} // Game()

public void setGUI( final UserInterface pUserInterface )
{
    this.aGui = pUserInterface;
    this.printWelcome();
}

private void printWelcome()
{
    this.aGui.print("\n");
    this.aGui.println("Welcome to the World of Pokemon!");
    this.aGui.println("A Wonderful World Where You Can Live An Adventure!");
    this.aGui.println("Type 'help' if you need help.");
    this.aGui.print("\n");
    this.printLocationInfo();
    if ( this.aCurrentRoom.getImageName() != null )
        this.aGui.showImage( this.aCurrentRoom.getImageName() );
} // printWelcome()
```

```java
private void createRooms()
{
    Room house, littleroot, route101, oldale, route102, petalburg, petalburgWoods, rustboro, skyPillar;

    house      = new Room("in your house in Littleroot Town","Littleroot_Town.jpg");
    littleroot = new Room("in your Littleroot, your Home Town","Littleroot_Town.jpg");
    route101   = new Room("on Route 101, wild Pokemon might appear in the tall grass","Littleroot_Town.jpg");
    oldale     = new Room("in Oldale Town, a small junction with a Pokemon Center","Littleroot_Town.jpg");
    route102   = new Room("on Route 102, wild Pokemon might appear in the tall grass","Littleroot_Town.jpg");
    petalburg  = new Room("in Petalburg City, where your father is the Gym Leader","Littleroot_Town.jpg");
    petalburgWoods  = new Room("in Petalburg Woods","Littleroot_Town.jpg");
    rustboro   = new Room("in Rustboro City, where the Devon Corporation is located","Littleroot_Town.jpg");
    skyPillar  = new Room("on the Sky Pilar, Rayquaza's home..","Sky Pillar.jpg");

    house.setExit("west", littleroot);

    littleroot.setExit("north", route101);
    littleroot.setExit("east", house);

    route101.setExit("north", oldale);
    route101.setExit("south", littleroot);

    oldale.setExit("east", route102);
    oldale.setExit("south", route101);

    route102.setExit("north", petalburgWoods);
    route102.setExit("east", petalburg);
    route102.setExit("west", oldale);

    petalburg.setExit("north", petalburgWoods);
    petalburg.setExit("west", route102);

    petalburgWoods.setExit("south", route102);
    petalburgWoods.setExit("west", rustboro);

    rustboro.setExit("up", skyPillar);
    rustboro.setExit("east", petalburgWoods);
    rustboro.setExit("south", oldale);

    skyPillar.setExit("down", rustboro);

    this.aCurrentRoom = house;
} // createRooms()
```

```java
public void processCommand (final String pCommand)
{
    this.aGui.println( "> " + pCommand );
    Command vCommand = this.aParser.getCommand( pCommand );

    if ( vCommand.isUnknown() ) {
        this.aGui.println( "I don't know what you mean..." );
        return;
    }

    String vCommandWord = vCommand.getCommandWord();
    if ( vCommandWord.equals( "help" ) )
        this.printHelp();
    else if ( vCommandWord.equals( "go" ) )
        this.goRoom( vCommand );
    else if ( vCommandWord.equals( "eat" ) )
        this.eat();
    else if ( vCommandWord.equals( "look" ) )
        this.look();
    else if ( vCommandWord.equals( "quit" ) ) {
        if ( vCommand.hasSecondWord() )
            this.aGui.println( "Quit what?" );
        else
            this.endGame();
    }
} // processCommand()
```

```java
private void endGame()
{
    this.aGui.println( "Thank you for playing.  Good bye." );
    this.aGui.enable( false );
}
```

```java
private void look()
{
   this.aGui.println(this.aCurrentRoom.getLongDescription());
} //look()
```

```java
/**
 * Prints a message indicating the player has eaten.
 */
private void eat()
{
    this.aGui.println("You have eaten now and you are not hungry any more.");
} //eat()
```

```java
private void printLocationInfo()
{
    this.aGui.print(this.aCurrentRoom.getLongDescription());
} // printLocationInfo()
```

```java
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.net.URL;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
```

Exercice 7.18.8:

Huit boutons cliquables ont été ajoutés à l'interface graphique (six pour les directions, un pour help, et un pour quit). Les boutons ont été instanciés, ajoutés à un JPanel avec un GridLayout, et reliés à un ActionListener (via la méthode actionPerformed) pour exécuter les commandes dans le moteur du jeu.

```java
private JButton    aButtonGoNorth;
private JButton    aButtonGoSouth;
private JButton    aButtonGoEast;
private JButton    aButtonGoWest;
private JButton    aButtonGoUp;
private JButton    aButtonGoDown;
private JButton    aButtonQuit;
private JButton    aButtonHelp;
```

```java
this.aButtonGoNorth = new JButton( "go north" );
this.aButtonGoSouth = new JButton( "go south" );
this.aButtonGoEast = new JButton( "go east" );
this.aButtonGoWest = new JButton( "go west" );
this.aButtonGoUp = new JButton( "go up" );
this.aButtonGoDown = new JButton( "go down" );
this.aButtonQuit = new JButton( "quit" );
this.aButtonHelp = new JButton( "help" );
```

```java
vPanel.add( vButtonPanel, BorderLayout.WEST );

JPanel vButtonPanel = new JPanel();
vButtonPanel.setLayout( new GridLayout( 8, 1 ) );
vButtonPanel.add( this.aButtonGoNorth );
vButtonPanel.add( this.aButtonGoSouth );
vButtonPanel.add( this.aButtonGoEast );
vButtonPanel.add( this.aButtonGoWest );
vButtonPanel.add( this.aButtonGoUp );
vButtonPanel.add( this.aButtonGoDown );
vButtonPanel.add( this.aButtonHelp );
vButtonPanel.add( this.aButtonQuit );

this.aEntryField.addActionListener( this );
this.aButtonGoNorth.addActionListener( this );
this.aButtonGoSouth.addActionListener( this );
this.aButtonGoEast.addActionListener( this );
this.aButtonGoWest.addActionListener( this );
this.aButtonGoDown.addActionListener( this );
this.aButtonGoUp.addActionListener( this );
this.aButtonHelp.addActionListener( this );
this.aButtonQuit.addActionListener( this );
```

```java
@Override public void actionPerformed( final ActionEvent pE )
{
    if( this.aButtonGoNorth.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go north");
    }
    else if( this.aButtonGoSouth.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go south");
    }
    else if( this.aButtonGoEast.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go east");
    }
    else if( this.aButtonGoWest.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go west");
    }
    else if( this.aButtonGoUp.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go up");
    }
    else if( this.aButtonGoDown.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("go down");
    }
    else if( this.aButtonQuit.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("quit");
    }
    else if( this.aButtonHelp.equals(pE.getSource()) )
    {
        this.aEngine.interpretCommand("help");
    }
    else {
        this.processCommand();
    }
```

Exercice 7.19.2:

Deplacement de toutes les images dans le sous dossier /images.

```java
Room house, littleroot, route101, oldale, route102, petalburg, petalburgWoods, rustboro, skyPillar;

house      = new Room("in your house in Littleroot Town","images/home.jpg");
littleroot = new Room("in your Littleroot, your Home Town","images/littleroot town.jpg");
route101   = new Room("on Route 101, wild Pokemon might appear in the tall grass","images/route 101.jpg");
oldale     = new Room("in Oldale Town, a small junction with a Pokemon Center","images/oldaletown.jpg");
route102   = new Room("on Route 102, wild Pokemon might appear in the tall grass","images/route 102.jpg");
petalburg  = new Room("in Petalburg City, where your father is the Gym Leader","images/petalburg city.jpg");
petalburgWoods  = new Room("in Petalburg Woods","images/petalburg woods.jpg");
rustboro   = new Room("in Rustboro City, where the Devon Corporation is located","images/rustboro city.jpg");
skyPillar  = new Room("on the Sky Pilar, Rayquaza's home..","images/sky pillar.jpg");
```

Exercice 7.19.2:

Les fichiers images ont été déplacés dans un sous-dossier dédié /images. La classe Item a été créée avec les attributs aDescription et aPrice. L'implémentation initiale dans la classe Room permettait de stocker un seul objet et la méthode getLongDescription a été retravaillée pour afficher cet objet s'il est présent.

```java
private String aDescription;
private int aPrice;

public Item (final String pDescription,final int pPrice)
{
    aDescription = pDescription;
    aPrice = pPrice;
}

public String getItemString()
{
    return this.aDescription;
} // getItemString()

public int getItemPrice()
{
    return this.aPrice;
} // getItemPrice()


private Item aItem;

public void setItem(final Item pItem)
{
    this.aItem = pItem;
}

public Item getItem(){
    return this.aItem;
}
```

```
house.setItem(new Item("map",0));
skyPillar.setItem(new Item("Delta Orb",100));
```

```
public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(this.aItem != null){
        vDescription = vDescription + '\n' + "There is " +
        this.aItem.getItemString() + '\n' + "Item price : " + this.aItem.getItemPrice();
    }
    else {
        vDescription = vDescription + '\n' + "there is no Items here";
    }
    return vDescription;
} // getLongDescription()
```

Exercice 7.21:

La méthode getItemString a été remplacée par getItemDescription pour renvoyer une description plus complète incluant le poids (ou prix) de l'objet, ce qui a été intégré dans getLongDescription.

```
public String getItemDescription()
{
    return "there is " + this.aDescription + "Item weight : " + this.aPrice;
}
```

```
public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(this.aItem != null){
        vDescription = vDescription + '\n' + this.aItem.getItemDescription();
    }
    else {
        vDescription = vDescription + '\n' + "there is no Items here";
    }
    return vDescription;
} // getLongDescription()
```

Exercice 7.22:

La classe Room a été modifiée pour utiliser une HashMap<String, Item> nommée aItems, permettant à une pièce de contenir plusieurs objets. La méthode getLongDescription itère désormais sur cette collection pour afficher tous les objets présents.

```
private HashMap<String, Item> aItems;
```

```
public Item getItem(final String pName){
    return this.aItems.get(pName);
}
```

```
public void addItem(final String pName, final Item pItem) {
    this.aItems.put(pName, pItem);
}
```

```
public String getLongDescription()
{
    String vDescription = "You are " + this.getDescription() + ".\n" + this.getExitString();
    if(!this.aItems.isEmpty()){
        for (String vName : this.aItems.keySet()) {
            vDescription += "\n" + this.aItems.get(vName).getItemDescription();
        }
    }
    else {
        vDescription += '\n' + "There are no Items here!";
    }
    return vDescription;
} // getLongDescription()
```

Exercice 7.22.2:

Plusieurs objets ont été instanciés au démarrage du jeu : une "Map" et des "Shoes" dans la maison (House), ainsi que la "Delta Orb" dans la pièce du Sky Pillar.

```
house.addItem("Map",new Item("The Map",0));
house.addItem("Shoes",new Item("Your Shoes",0));
skyPillar.addItem("Delta Orb",new Item("The Delta Orb, the orb that will allow you to summon Rayquazza..",100));
```

Exercice 7.23:

Ajout de la commande back permettant au joueur de revenir sur ses pas. Un historique des déplacements a d'abord été implémenté avec une LinkedList<Room> (aPreviousRoom). Des conditions ont été ajoutées pour empêcher le joueur de faire "back" s'il vient de commencer.

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back" };
```

```
private LinkedList<Room> aPreviousRoom = new LinkedList<Room>();
```

```
this.aPreviousRoom.add(null);
```

```
private void goRoom (final Command pCommand)
{
    if ( ! pCommand.hasSecondWord() ) {
        // if there is no second word, we don't know where to go...
        this.aGui.println( "Go where?" );
        return;
    }

    String vDirection = pCommand.getSecondWord();

    // Try to leave current room.
    Room vNextRoom = this.aCurrentRoom.getExit( vDirection );

    if ( vNextRoom == null )
        this.aGui.println( "There is no door!" );
    else {
        this.aPreviousRoom.addLast(this.aCurrentRoom);
        this.aCurrentRoom = vNextRoom;
        this.aGui.println( this.aCurrentRoom.getLongDescription() );
        if ( this.aCurrentRoom.getImageName() != null )
            this.aGui.showImage( this.aCurrentRoom.getImageName() );
    }
} // goRoom()
```

```
    else if ( vCommandWord.equals( "back" ) )
        if ( vCommand.hasSecondWord() )
            this.aGui.println( "Back what?" );
        else
            if (this.aPreviousRoom.getLast() != null)
                this.goBack();
            else
                this.aGui.println( "Back where? You just Started!" );
```

```
private void goBack()
{
    this.aCurrentRoom = this.aPreviousRoom.getLast();
    this.aPreviousRoom.removeLast();
    this.printLocationInfo();
    this.aGui.showImage( this.aCurrentRoom.getImageName());
}
```

Exercice 7.26:

La LinkedList a été remplacée de manière plus judicieuse par une pile Stack<Room> (aPreviousRooms), et les méthodes goRoom et goBack ont été mises à jour pour utiliser push() et pop().

```
private Stack<Room> aPreviousRooms = new Stack<Room>();
```

```
        this.aPreviousRooms.push(this.aCurrentRoom);
```

```
            if (!this.aPreviousRooms.isEmpty())
```

```
private void goBack()
{
    this.aCurrentRoom = this.aPreviousRooms.pop(); // retrieves AND removes the top
    this.printLocationInfo();
    this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

Exercice 7.28.1:

Une commande test a été ajoutée pour lire et exécuter automatiquement une suite de commandes depuis un fichier texte (avec l'extension .txt). Cette fonctionnalité s'appuie sur InputStream et Scanner pour lire les lignes et les envoyer à interpretCommand.

```java
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test" };

import java.io.InputStream;
import java.util.Scanner;
```

```java
else if (vCommandWord.equals("test"))
    if(vCommand.hasSecondWord() == false)
        this.aGui.println("Need a file name!");
    else
        test(vCommand);
```

```java
private void test(final Command pCommand)
{
    String vFileName = pCommand.getSecondWord();
    vFileName = vFileName + ".txt";

    InputStream vIS = this.getClass().getClassLoader().getResourceAsStream(vFileName);

    if (vIS == null)
    {
        System.out.println("File not found: " + vFileName);
        return;
    }

    Scanner vSC = new Scanner(vIS);

    while (vSC.hasNextLine())
    {
        String vLigne = vSC.nextLine();
        this.interpretCommand(vLigne);
    }

    vSC.close();
}
```

Exercice 7.29:

Création de la classe Player pour encapsuler les données du joueur, notamment son nom (aName), sa position actuelle (aCurrentRoom), et son historique de pièces (aPreviousRooms). Le GameEngine a été adapté en conséquence et une commande name a été introduite pour permettre au joueur de se renommer.

```java
private String aName;
private Room aCurrentRoom;
private Stack<Room> aPreviousRooms;

/**
 * Create a player with a name and a starting room.
 * @param pName the player name (can be empty or null)
 * @param pStartRoom the initial room
 */
public Player(final String pName, final Room pStartRoom)
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
}

/**
 * Return the player's current room.
 */
public Room getCurrentRoom()
{
    return this.aCurrentRoom;
}
```

```java
/**
 * Change the current room, saving the previous one.
 * This is called by the GameEngine after it validates the direction.
 * @param pNextRoom the room to move to
 */
public void moveTo(final Room pNextRoom)
{
    if (this.aCurrentRoom != null) {
        this.aPreviousRooms.push(this.aCurrentRoom);
    }
    this.aCurrentRoom = pNextRoom;
}

/**
 * Return true if the player can go back to a previous room.
 */
public boolean canGoBack()
{
    return !this.aPreviousRooms.isEmpty();
}

/**
 * Move back to the previous room if possible.
 * @return the new current room (or current room if no previous room)
 */
public void goBack()
{
    this.aCurrentRoom = this.aPreviousRooms.pop();
}

/**
 * Return the player name.
 */
public String getName()
{
    return this.aName;
}

/**
 * Set the player name.
 */
public void setName(final String pName)
{
    this.aName = pName;
}
```

```java
private void goRoom( final Command pCommand )
{
    if ( ! pCommand.hasSecondWord() ) {
        // If there is no second word, we don't know where to go.
        this.aGui.println( "Go where?" );
        return;
    }

    String vDirection = pCommand.getSecondWord();

    Room vNextRoom = this.aPlayer.getCurrentRoom().getExit( vDirection );

    if ( vNextRoom == null )
        this.aGui.println( "There is no door!" );
    else {
        this.aPlayer.moveTo(vNextRoom);
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
        if ( this.aPlayer.getCurrentRoom().getImageName() != null )
            this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    }
} // goRoom()
```

```java
private void goBack( final Command pCommand )
{
    if ( pCommand.hasSecondWord() ) {
        // If there is a second word, we don't know where to go.
        this.aGui.println( "Back where?" );
        return;
    }

    if (!this.aPlayer.canGoBack())
        this.aGui.println( "Can't go back, you just started!" );
    else {
        this.aPlayer.goBack();
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
        if ( this.aPlayer.getCurrentRoom().getImageName() != null )
            this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    }
} // goRoom()
```

```java
this.aPlayer = new Player("Luke", house);
```

```java
else if (vCommandWord.equals("name"))
    if(vCommand.hasSecondWord() == false)
        this.aGui.println("You need a name!");
    else
        name(vCommand);
```

```java
private void name(final Command pCommand)
{
    String vName = pCommand.getSecondWord();
    this.aPlayer.setName(vName);
    this.aGui.println( "Your new name is " + this.aPlayer.getName() + '!' );
    this.printWelcome();
} // name()
```

Exercice 7.30:

Ajout des commandes take et drop. Le joueur pouvait initialement ramasser et stocker un seul objet via des méthodes setItem et removeItem.

```
private String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test", "name", "take", "drop"};


private Item aItem;

public void setItem(final Item pItem)
{
    this.aItem = pItem;
}

public void removeItem(final String pName){
    this.aItems.remove(pName);
}

public Item getItem()
{
    return this.aItem;
}

public void removeItem(){
    this.aItem = null;
}

private void take(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false)
    {
        this.aGui.println("Take what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);

    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.setItem(vItem);
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
    this.aGui.println("Took " + vItemName + "!");
}

private void drop (final Command pCommand){
    if(pCommand.hasSecondWord() == false){
        this.aGui.println("Drop what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getItem();
    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem();
    this.aGui.println("Dropped " + vItemName + "!");
}

else if ( vCommandWord.equals( "take" ) )
    this.take(vCommand);
else if ( vCommandWord.equals( "drop" ) )
    this.drop(vCommand);
```

Exercice 7.31:

Le joueur est désormais capable de transporter plusieurs objets, son inventaire étant géré par une HashMap<String, Item>. Les méthodes take et drop ont été mises à jour pour utiliser addItem et removeItem à la fois pour le joueur et la pièce.

```
private HashMap<String, Item> aItems;

public Player(final String pName, final Room pStartRoom)
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new HashMap<String, Item>();
}
```

```java
public Item getItem(final String pNom)
{
    return this.aItems.get(pNom);
}

public void addItem(final String pNom, final Item pItem)
{
    this.aItems.put(pNom, pItem);
}

public void removeItem(final String pNom)
{
    this.aItems.remove(pNom);
}

private void take(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false)
    {
        this.aGui.println("Take what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);

    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().removeItem(vItemName);
    this.aPlayer.addItem(vItemName, vItem);
    this.aGui.println("Took " + vItemName + "!");
}

private void drop(final Command pCommand)
{
    if(pCommand.hasSecondWord() == false){
        this.aGui.println("Drop what?");
        return;
    }
    String vItemName = pCommand.getSecondWord();
    Item vItem = this.aPlayer.getItem(vItemName);
    if(vItem == null)
    {
        this.aGui.println("I can't find any " + vItemName + "!");
        return;
    }
    this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
    this.aPlayer.removeItem(vItemName);
    this.aGui.println("Dropped " + vItemName + "!");
}
```

Exercice 7.31.1:

Pour éviter la duplication de code entre Player et Room, la classe ItemList a été créée. Elle encapsule la logique de gestion de la HashMap des objets (addItem, getItem, removeItem, getItemsDescription). Les classes Room et Player l'utilisent désormais comme attribut (aItems).

```java
private HashMap<String, Item> aItems;

/**
 * Creates an empty item list.
 */
public ItemList()
{
    this.aItems = new HashMap<String, Item>();
}

/**
 * Returns the item associated with the given name, or {@code null}.
 *
 * @param pName the lookup key
 * @return the matching {@link Item}, or {@code null}
 */
public Item getItem( final String pName )
{
    return this.aItems.get( pName );
}

/**
 * Adds or replaces an item under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pItem the {@link Item} to store
 */
public void addItem( final String pName, final Item pItem )
{
    this.aItems.put( pName, pItem );
}

/**
 * Removes the item associated with the given key.
 *
 * @param pName the lookup key
 */
public void removeItem( final String pName )
{
    this.aItems.remove( pName );
}

/**
 * Returns {@code true} if the list contains no items.
 */
public boolean isEmpty()
{
    return this.aItems.isEmpty();
}

/**
 * Builds a multi-line string with each item's description,
 * or a default notice when the list is empty.
 *
 * @return formatted item descriptions
 */
public String getItemsDescription()
{
    if ( this.aItems.isEmpty() ) {
        return "There are no items here!";
    }
    String vResult = "";
    for ( String vName : this.aItems.keySet() ) {
        vResult +=  this.aItems.get( vName ).getItemDescription() + "\n" ;
    }
    return vResult;
}
```

```java
private ItemList aItems;
/**
 * Creates a room with a description, image path, and empty exit/item maps.
 *
 * @param pDescription human-readable location phrase shown to the player
 * @param pImage       resource path for the picture, or {@code null} if none
 */
public Room( final String pDescription, final String pImage )
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
    this.aItems = new ItemList();
} // Room()

public Item getItem( final String pName )
{
    return this.aItems.getItem( pName );
}

/**
 * Places (or replaces) an item in this room under the given key.
 *
 * @param pName short identifier used for later retrieval
 * @param pItem the {@link Item} instance to store
 */
public void addItem( final String pName, final Item pItem )
{
    this.aItems.addItem( pName, pItem );
}

public void removeItem(final String pName){
    this.aItems.removeItem(pName);
}
```

```java
private ItemList aItems;
```

```java
/**
 * Create a player with a name and a starting room.
 * @param pName the player name (can be empty or null)
 * @param pStartRoom the initial room
 */
public Player(final String pName, final Room pStartRoom)
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new ItemList();
```

```java
public Item getItem(final String pNom)
{
    return this.aItems.getItem(pNom);
}
```

```java
public void addItem(final String pNom, final Item pItem)
{
    this.aItems.addItem(pNom, pItem);
}
```

```java
public void removeItem(final String pNom)
{
    this.aItems.removeItem(pNom);
}
```

Bonus :

Pour mettre en place la fin du jeu, un attribut booléen isWinningRoom a été ajouté pour déterminer si une pièce spécifique est la salle déclenchant la victoire. Des méthodes d'accès ont été intégrées pour manipuler et vérifier cet état.

Modification de la classe Room : Ajout d'un setter pour définir la pièce comme gagnante et d'un booléen public (getter) pour vérifier son état.

Définition de l'objectif : Lors de la création des pièces, le Sky Pillar est défini comme la salle de victoire.

Vérification dans le GameEngine : Une condition a été ajoutée pour vérifier si le joueur se trouve dans la winning room tout en possédant l'objet requis ("Delta Orb"). Si ces conditions sont remplies, le joueur gagne la partie.

```java
public void setAsWinningRoom()
{
    this.isWinningRoom = true;
}
```

```java
public boolean isWinningRoom()
{
    return this.isWinningRoom;
}
```

```java
skyPillar.setAsWinningRoom();
```

```java
        if ( this.aPlayer.getItem( "delta-orb" ) != null ) {
            if ( this.aPlayer.getCurrentRoom().isWinningRoom() ) {
                this.aGui.println( "\n" + "Congratulations! You just won! Thank you for playing the whole game!" );
            }
        }
    }
```

Un système d'affichage de carte évolutif a été ajouté, permettant au joueur de voir sa position à condition d'avoir préalablement ramassé l'objet "carte".Adaptation de la classe Room : Le constructeur de la pièce accepte désormais un paramètre supplémentaire pour lier une image de carte spécifique (aMapImageName) au lieu. Chaque pièce est désormais instanciée avec son image principale et l'image de sa carte (ex: images/home_map.jpeg).Remaniement de l'interface graphique (UserInterface) :Mise en page de l'image : Création d'un nouveau panel exclusif à l'image principale pour la maintenir centrée en utilisant setHorizontalAlignment et setVerticalAlignment.Zone de navigation : Les boutons de déplacement et d'action (incluant le nouveau bouton back) ont été organisés sous forme de grille 3x3 (GridLayout(3, 3)).Zone de la carte : L'affichage de la carte a été assigné à la position est (BorderLayout.EAST) du panneau

inférieur (SouthPanel).Logique interactive dans le GameEngine :Le système vérifie si le joueur possède la carte lors des déplacements (goRoom, goBack) et lorsqu'il ramasse des objets (take) afin d'actualiser l'affichage instantanément.Si le joueur a la carte, la méthode affiche l'image de la carte associée à la pièce actuelle.Si le joueur ne possède pas ou lâche la carte (drop), une image par défaut (ex: images/no_map.jpeg) est affichée à la place.

```
private String aMapImageName;
```

```
public Room( final String pDescription, final String pImage, final String pMapImage )
{
    this.aDescription = pDescription;
    this.aExits = new HashMap<String, Room>();
    this.aImageName = pImage;
    this.aMapImageName = pMapImage;
    this.aItems = new ItemList();
} // Room()
```

```
house      = new Room( "in your house in Littleroot Town", "images/home.gif", "images/home_map.jpeg" );
littleroot = new Room( "in Littleroot Town, your home town", "images/littleroot town.gif", "images/littleroot_map.jpeg" );
route101   = new Room( "on Route 101; wild Pokémon might appear in the tall grass", "images/route 101.gif", "images/route101_map.jpeg" );
oldale     = new Room( "in Oldale Town, a small junction with a Pokémon Center", "images/oldale town.gif", "images/oldale town_map.jpeg" );
route102   = new Room( "on Route 102; wild Pokémon might appear in the tall grass", "images/route 102.gif", "images/route102_map.jpeg" );
petalburg  = new Room( "in Petalburg City, where your father is the Gym Leader", "images/petalburg city.gif", "images/petalburg city_map.jpeg" );
petalburgWoods  = new Room( "in Petalburg Woods", "images/petalburg woods.gif", "images/petalburg woods_map.jpeg" );
rustboro   = new Room( "in Rustboro City, where the Devon Corporation is located", "images/rustboro city.gif", "images/rustboro city_map.jpeg" );
skyPillar  = new Room( "on the Sky Pillar, Rayquaza's home", "images/sky pillar.gif", "images/sky pilar_map.jpeg" );
```

```
this.aImage = new JLabel();
this.aImage.setHorizontalAlignment( JLabel.CENTER );
this.aImage.setVerticalAlignment( JLabel.CENTER );

this.aMap = new JLabel();

this.aButtonGoUp = new JButton( "go up" );
this.aButtonGoNorth = new JButton( "go north" );
this.aButtonGoDown = new JButton( "go down" );
this.aButtonGoWest = new JButton( "go west" );
this.aButtonGoEast = new JButton( "go east" );
this.aButtonBack = new JButton( "go back" );
this.aButtonHelp = new JButton( "help" );
this.aButtonGoSouth = new JButton( "go south" );
this.aButtonQuit = new JButton( "quit" );

JPanel vButtonPanel = new JPanel();
vButtonPanel.setLayout( new GridLayout( 3, 3 ) );
vButtonPanel.add( this.aButtonGoUp );
vButtonPanel.add( this.aButtonGoNorth );
vButtonPanel.add( this.aButtonGoDown );
vButtonPanel.add( this.aButtonGoWest );
vButtonPanel.add( this.aButtonBack );
vButtonPanel.add( this.aButtonGoEast );
vButtonPanel.add( this.aButtonHelp );
vButtonPanel.add( this.aButtonGoSouth );
vButtonPanel.add( this.aButtonQuit );

JPanel vPanel = new JPanel();
vPanel.setLayout( new BorderLayout() );
vPanel.add( this.aImage, BorderLayout.CENTER );

JPanel vSouthPanel = new JPanel();
vSouthPanel.setLayout( new BorderLayout() );
vSouthPanel.setPreferredSize( new Dimension( 0, 400 ) );

vSouthPanel.add( vListScroller, BorderLayout.CENTER );
vSouthPanel.add( this.aEntryField, BorderLayout.SOUTH );
vSouthPanel.add( vButtonPanel, BorderLayout.WEST );
vSouthPanel.add( this.aMap, BorderLayout.EAST );

vPanel.add(vSouthPanel, BorderLayout.SOUTH);

this.aMyFrame.getContentPane().add( vPanel, BorderLayout.CENTER );

this.aEntryField.addActionListener( this );
this.aButtonGoNorth.addActionListener( this );
this.aButtonGoSouth.addActionListener( this );
this.aButtonGoEast.addActionListener( this );
this.aButtonGoWest.addActionListener( this );
this.aButtonGoUp.addActionListener( this );
this.aButtonGoDown.addActionListener( this );
this.aButtonBack.addActionListener( this );
this.aButtonQuit.addActionListener( this );
this.aButtonHelp.addActionListener( this );
```

```
if ( this.aPlayer.getCurrentRoom().getImageName() != null ) {
    this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
    if ( this.aPlayer.getItem( "map" ) != null ) {
        this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
    }
}
```

```
if ( this.aPlayer.getItem( "map" ) != null ) {
    this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
}
```

```
if ( this.aPlayer.getItem( "map" ) == null ) {
    this.aGui.showMap( "images/no map.jpeg" );
}
```

Exercice 7.32:

Un système de monnaie a été mis en place, rendant l'argent nécessaire pour l'obtention des objets.Classe Player : Un nouvel attribut entier aMoney a été créé pour représenter le solde du joueur. Cet attribut s'accompagne de ses méthodes d'accès, à savoir un accesseur getMoney() et un mutateur setMoney(). Par défaut, le joueur commence avec 150 unités de monnaie. Classe GameEngine : Ce système a été implémenté au sein des méthodes take et drop. Lors de la saisie d'un objet via take, le moteur vérifie si le joueur possède un montant supérieur au prix de l'objet ; si c'est le cas, l'objet est ajouté à l'inventaire et le montant est déduit du solde du joueur. À l'inverse, lorsqu'un objet est relâché, le prix de ce dernier est restitué au joueur. S'il n'a pas les fonds nécessaires, un message indique que l'objet est trop cher.

```java
private int aMoney;

/**
 * Creates a player with a name and a starting room.
 *
 * @param pName      the player name (can be empty or {@code null})
 * @param pStartRoom the initial room
 */
public Player( final String pName, final Room pStartRoom )
{
    this.aName = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aItems = new ItemList();
    this.aMoney = 150;
}
```

```java
public int getMoney()
{
    return this.aMoney;
}
```

```java
public void setMoney( final int pNewMoney )
{
    this.aMoney = pNewMoney;
}
```

```java
if ( this.aPlayer.getMoney() >= vItem.getItemPrice() )
{
    this.aPlayer.setMoney( this.aPlayer.getMoney() - vItem.getItemPrice() );
    this.aPlayer.getCurrentRoom().removeItem( vItemName );
    this.aPlayer.addItem( vItemName, vItem );
    this.aGui.println( "\nTook " + vItemName + "!" );
    if ( this.aPlayer.getItem( "map" ) != null ) {
        this.aGui.showMap( this.aPlayer.getCurrentRoom().getMapImageName() );
    }
} else
{
    this.aGui.println( vItemName + " is too expensive for you right now !" );
}
```

```java
this.aPlayer.setMoney( this.aPlayer.getMoney() + vItem.getItemPrice() );
```

Exercice 7.33:

La possibilité pour le joueur de consulter les objets qu'il transporte a été ajoutée via la nouvelle commande inventory.Déclaration : Le mot-clé "inventory" a été ajouté au tableau aValidCommands dans la classe CommandWords.Classe ItemList : La méthode getItemList() a été créée pour générer une chaîne de caractères listant les objets du joueur, en précisant qu'il n'a rien si sa liste est vide. Cette méthode calcule également la valeur totale (vInventoryPrice) des objets possédés.Liaison et Affichage : Une méthode getItemInventory() a été mise en place pour relayer cette information vers la classe Player. Enfin, la méthode inventory() dans le GameEngine se charge d'afficher ce message à l'écran.

```java
private String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test", "name", "take", "drop", "inventory" };
```

```java
public String getItemList()
{
    if ( this.aItems.isEmpty() ) {
        return "You don't have any items!";
    }
    String vResult = "Your items: ";
    int vInventoryPrice = 0;
    for ( String vItem : this.aItems.keySet() ) {
        vResult += " " + vItem;
        vInventoryPrice += this.aItems.get( vItem ).getItemPrice();
    }
    vResult += "\n" + "\n" + "Total value: " + vInventoryPrice;
    return vResult;
}
```

```java
public String getItemInventory()
{
    return this.aItems.getItemList() ;
}
```

```java
private void inventory()
{
    this.aGui.println( this.aPlayer.getItemInventory() );
}
```

Exercice 7.34:

Le jeu intègre désormais un moyen de gagner de l'argent en convertissant certains objets.Refonte de la commande : L'ancienne commande eat a été transformée en la commande cashin dans les CommandWords.Méthode cashin dans GameEngine : La méthode d'origine a été remplacée en profondeur par cashin(final Command pCommand).Objets convertibles : Une HashMap<String, Integer> nommée vCashableItem a été mise en place pour stocker plusieurs objets utilisables pour obtenir de l'argent et leurs valeurs de récompense associées. Par exemple, l'objet "grant" rapporte 75 unités de monnaie, et le "wallet" en rapporte 25.Logique de conversion : Si le joueur utilise la commande avec un objet valide et qu'il le possède dans son inventaire, l'objet est supprimé et l'argent est crédité sur son solde (setMoney(this.aPlayer.getMoney() + vReward)). Un message confirme ensuite la transaction et affiche le nouveau solde.

```java
private void cashin( final Command pCommand )
{
    String vItemName = pCommand.getSecondWord();
    HashMap<String, Integer> vCashableItem = new HashMap<String, Integer>();
    vCashableItem.put( "grant", 75 );
    vCashableItem.put( "wallet", 25 );

    if ( vCashableItem.containsKey( vItemName ) ) {
        Item vItem = this.aPlayer.getItem( vItemName);
        if (vItem != null) {
            int vReward = vCashableItem.get( vItemName );
            this.aPlayer.setMoney(this.aPlayer.getMoney() + vReward);
            this.aPlayer.removeItem( vItemName );
            this.aGui.println("You have cashed in your " + vItemName + " and are now richer!");
            this.aGui.println("Your current money is: " + this.aPlayer.getMoney());
        } else {
            this.aGui.println("You don't have a " + vItemName + " to cash in.");
        }
    } else {
        this.aGui.println("You can't cash that in.");
    }
} // cashin()
```

0

III. Guide Utilisateur

Pour jouer au jeux:

Créer une instance de Game.

Utiliser les commandes : go [direction](north; east; south; west; up; down) , back, take [item], drop [item], name [new name], test [test filename]. look, cashin [item], help, quit, inventory.

IV. Déclaration Anti-Plagiat

Moi, Barnabé Jouanard, déclare n'avoir copié aucune ligne de code d'une source externe